

[TA] [АСИНХРОН] Интеграция с Unisender для отправки уведомлений

Общее описание

Интеграция с Unisender для отправки email и SMS уведомлений об успешной покупке курса.

Ссылка на отправку уведомления генерируется асинхронно через сервис-адаптер к Unisender и Kafka.

API-документация Внешней системы

 [Интеграция сервиса для рассылок Unisender по API](#)

Ограничения Внешней системы:

- **Лимит запросов:** 100 запросов в минуту (согласно документации Unisender)
- **Максимальная длина SMS:** 160 символов (для одного сообщения)
- **Формат email:** стандартный HTML + текст
- **Таймаут:** 30 секунд
- **Требуется авторизация:** API-ключ в каждом запросе

Сценарий работы (Use Case)

Роли пользователей:

- Пользователь (ученик) - получает уведомление
- Система U-Verse (Payment Service, Notification Service)

Приложения и системы:

- Веб-приложение (личный кабинет пользователя)
- Backend-приложение (Payment Service)
- Backend-приложение (Notification Service)
- База данных (БД) U-Verse
- Брокер сообщений (Kafka)
- Внешняя система: API Unisender

Входные данные:

- Идентификатор пользователя (`userId`)
- Email пользователя (`email`)
- Номер телефона пользователя (`phone` — опционально)

- Идентификатор заказа (`orderId`)
- Название купленного курса (`courseName`)
- Стоимость курса (`coursePrice`)

Предусловие:

- Пользователь авторизован в системе
- Пользователь успешно оплатил курс
- Платеж подтвержден платежной системой
- В системе настроена интеграция с Unisender (API-ключ, шаблоны уведомлений)
- Настроены топики Kafka для обмена сообщениями

Алгоритм работы:

1. Пользователь оплачивает курс.
 - 1.1. Пользователь выбирает курс и переходит к оплате через платежную систему.
 - 1.2. Платежная система (например, Т-Банк) подтверждает успешную оплату.
2. Payment Service обрабатывает успешный платеж.
 - 2.1. Payment Service получает уведомление об успешной оплате от платежной системы.
 - 2.2. Payment Service сохраняет информацию о платеже в БД (статус `PAID` , сумма, дата).
 - 2.3. Payment Service формирует событие `PAYMENT_SUCCESS` и публикует его в топик Kafka `payment_notifications` .
 - 2.4. Payment Service не ожидает ответа от Notification Service (асинхронный режим).
3. Notification Service обрабатывает событие.
 - 3.1. Notification Service (Consumer) вычитывает сообщение из топика `payment_notifications` .
 - 3.2. Notification Service валидирует полученные данные (проверяет наличие обязательных полей: `email` , `userId` , `courseName`).
 - 3.3. Notification Service формирует шаблон email-уведомления (приветствие, название курса, благодарность за покупку).
 - 3.4. Notification Service отправляет запрос к API Unisender для отправки email:

1

POST <https://api.unisender.com/ru/api/sendEmail>
 - 3.5. Unisender обрабатывает запрос и возвращает результат.

3.6. При успешной отправке email Notification Service проверяет наличие номера телефона в сообщении.

3.7. Если номер телефона указан, Notification Service формирует SMS-уведомление и отправляет запрос к API Unisender:

```
1 | POST https://api.unisender.com/ru/api/sendSms
```

3.8. Notification Service сохраняет статус отправки в БД (успех/ошибка).

3.9. Notification Service подтверждает (commit) обработанное сообщение в Kafka.

4. Пользователь получает уведомление.

4.1. На email пользователя приходит письмо с подтверждением покупки курса.

4.2. На телефон пользователя (если указан) приходит SMS-уведомление о покупке.

Альтернативные сценарии и обработка ошибок

Недоступность Kafka (п.2.3)

Payment Service не может опубликовать сообщение в Kafka. Payment Service выполняет повторные попытки (до 3 раз с задержкой). При неудаче логирует ошибку с уровнем CRITICAL и уведомляет администратора.

Ошибка валидации данных (п.3.2)

Notification Service получил сообщение с отсутствием обязательных полей. Notification Service логирует ошибку с уровнем ERROR, сохраняет статус **FAILED** в БД. Сообщение в Kafka не подтверждается (не commit), чтобы оно могло быть обработано повторно после исправления ошибки.

Ошибка отправки email через Unisender (п.3.4)

API Unisender вернул ошибку (5xx, таймаут, неверный формат). Notification Service выполняет повторные попытки с экспоненциальной задержкой (до 5 попыток). При успехе после retry — сохраняет статус **SUCCESS**. При исчерпании всех попыток — сохраняет статус **FAILED**, логирует ошибку, сообщение остается в Kafka для ручного анализа.

Недоступность Unisender (п.3.4)

API Unisender недоступно (ошибка 503, таймаут). Notification Service выполняет повторные попытки с экспоненциальной задержкой. При длительной недоступности сообщение остается в Kafka для повторной обработки позже.

Ошибка отправки SMS через Unisender (п.3.7)

API Unisender вернул ошибку при отправке SMS. Notification Service выполняет повторные попытки для SMS (аналогично email). Если SMS не отправлена, но email отправлен успешно — сохраняет статус `PARTIAL_SUCCESS` (частичный успех).

Ошибка сохранения статуса в БД (п.3.8)

Notification Service не может сохранить статус отправки в БД. Notification Service логирует ошибку с уровнем ERROR. Сообщение в Kafka не подтверждается (не commit), чтобы сохранить возможность повторной обработки.

Истек срок жизни сообщения (TTL)

Сообщение в Kafka не было обработано в течение 7 дней (TTL). Сообщение автоматически удаляется из топика. Администратор получает уведомление о необходимости ручной отправки уведомления пользователю.

Frontend

[\[TA Web-приложение\] Frontend Отображение статуса покупки и уведомлений](#)

Техническая реализация Backend

[\[TA\] \[МС Платежей\] Обработка успешного платежа и публикация события](#)

[\[TA\] \[МС Уведомлений\] Worker для обработки событий отправки уведомлений](#)

[\[TA\] \[МС Уведомлений\] Интеграционный метод для отправки уведомлений](#)

Kafka

[\[TA\] \[Kafka\] Топики для уведомлений](#)

Логирование

- Логировать все входящие события из Kafka
- Логировать запросы к API Unisender и полученные ответы
- Логировать ошибки с уровнем ERROR
- Логировать успешные отправки с уровнем INFO
- Формат лога: `timestamp, level, service, userId, orderId, channel, status, message`